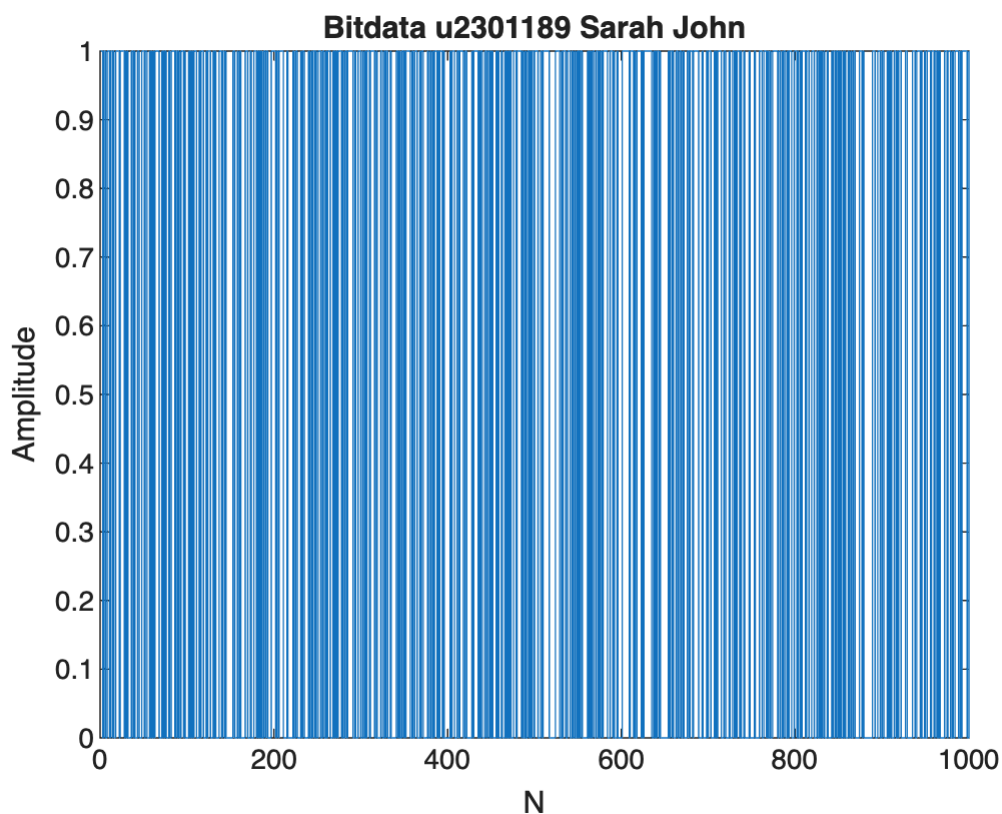


```
clc
clear all
close all
N=input("Enter the number of bits:");
bit=randi([0,1],1,N)
```

```
bit = 1×1000
      1      1      0      1      1      0      0      1      1      1      0      1      1...
```

```
stairs(bit)
xlabel("N");
ylabel("Amplitude");
title("Bitdata u2301189 Sarah John");
```



```
A=input("Enter the amplitude:");
n=input("Enter the number of samples:");
```

Signal Generation

```
t1=linspace(0,4*pi,n);
c1=A*sin(t1);
c_t1=transpose(c1);
E1=c1*c_t1;
cnorm1=c1/sqrt(E1);
t2=linspace(0,4*pi,n);
```

```

c2=A*cos(t2);
c_t2=transpose(c2);
E2=c2*c_t2;
cnorm2=c2/sqrt(E2);

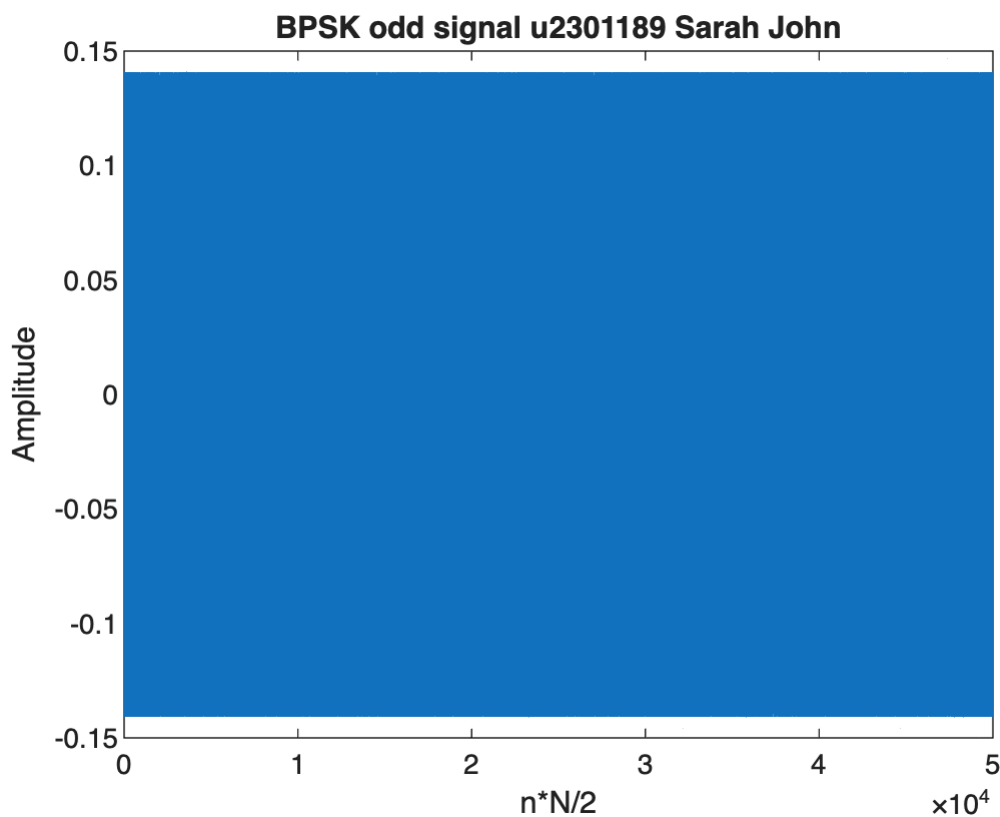
```

QPSK modulation

```

bpsk_odd=[];%cosine
b_odd=bit(1:2:N);
for i=1:N/2
    if b_odd(i)==1
        bpsk_odd=[bpsk_odd cnorm2];
    elseif b_odd(i)==0
        bpsk_odd=[bpsk_odd cnorm2*-1];
    end
end
plot(bpsk_odd)
xlabel("n*N/2");
ylabel("Amplitude");
title("BPSK odd signal u2301189 Sarah John");

```



```

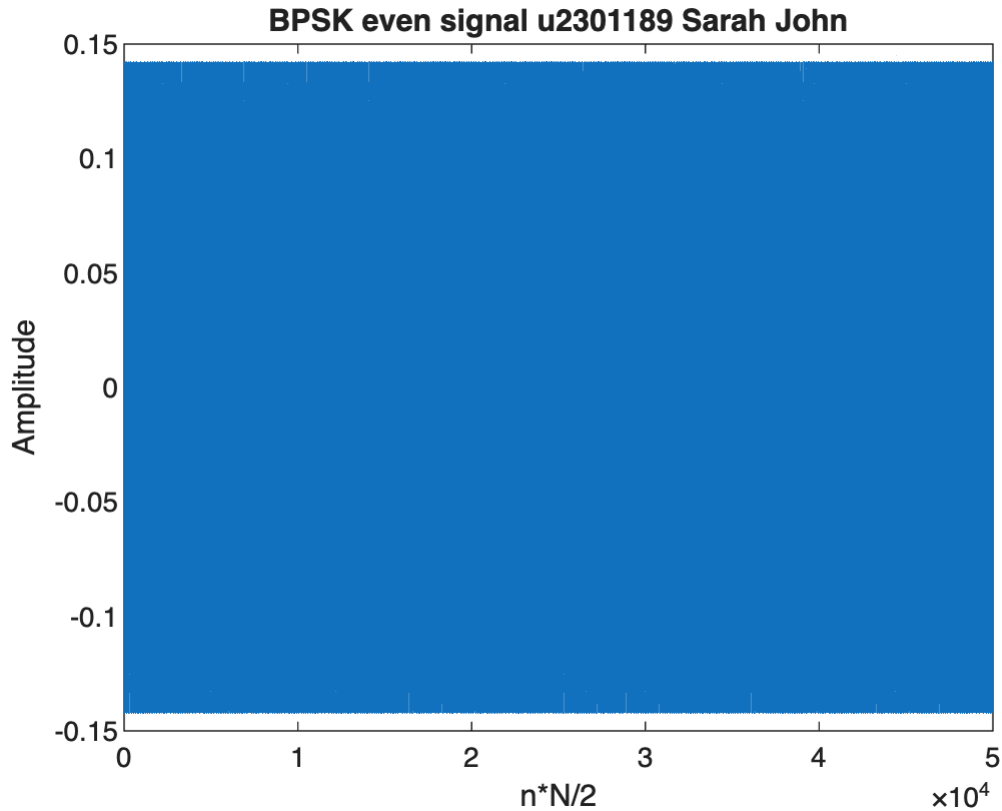
bpsk_even=[];%sine
b_even=bit(2:2:N);
for i=1:N/2
    if b_even(i)==1
        bpsk_even=[bpsk_even cnorm1];
    elseif b_even(i)==0

```

```

bpsk_even=[bpsk_even cnorm1*-1];
end
end
plot(bpsk_even)
xlabel("n*N/2");
ylabel("Amplitude");
title("BPSK even signal u2301189 Sarah John");

```

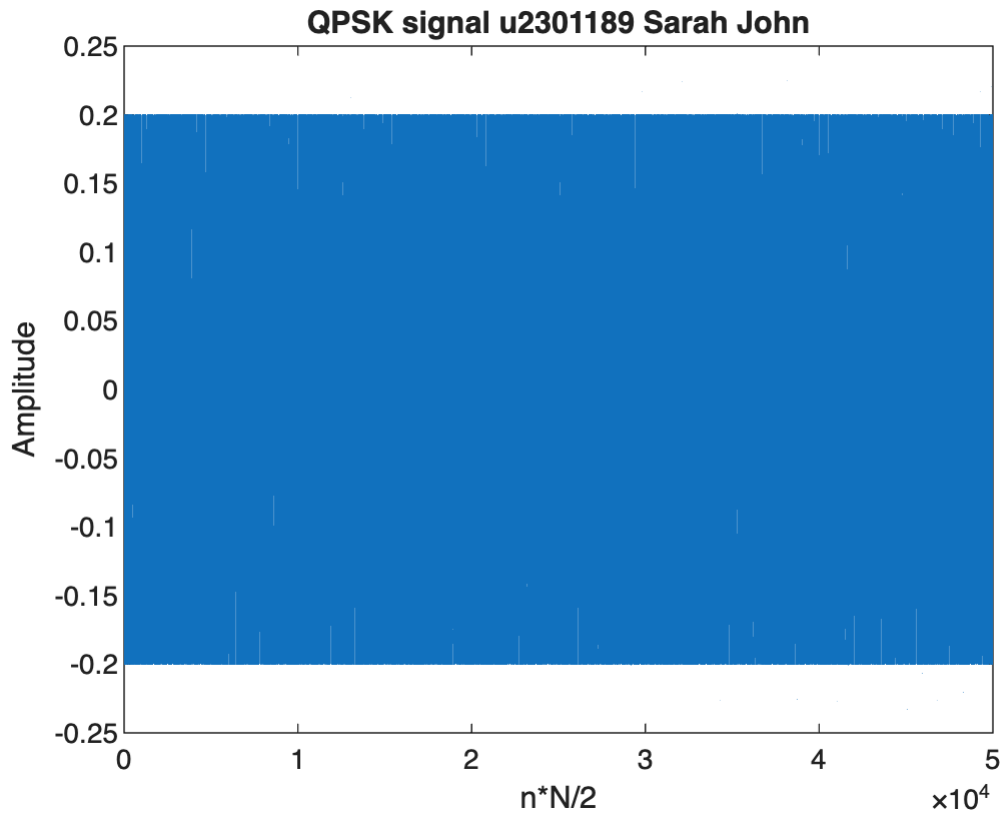


Demodulation

```

qpsk=bpsk_even+bpsk_odd;
plot(qpsk)
xlabel("n*N/2");
ylabel("Amplitude");
title("QPSK signal u2301189 Sarah John");

```



```

rb1=reshape(qpsk,[n,N/2]);
product1=cnorm1*rb1;
deconst1=[];
for j=1:N/2
if product1(j)>0
deconst1=[deconst1 1];
elseif product1(j)<0
deconst1=[deconst1 0];
end
end
rb2=reshape(qpsk,[n,N/2]);
product2=cnorm2*rb2;
deconst2=[];
for j=1:N/2
if product2(j)>0
deconst2=[deconst2 1];
elseif product2(j)<0
deconst2=[deconst2 0];
end
end
demod=[];
for j=1:N/2
demod=[demod deconst2(j) deconst1(j)];
end

```

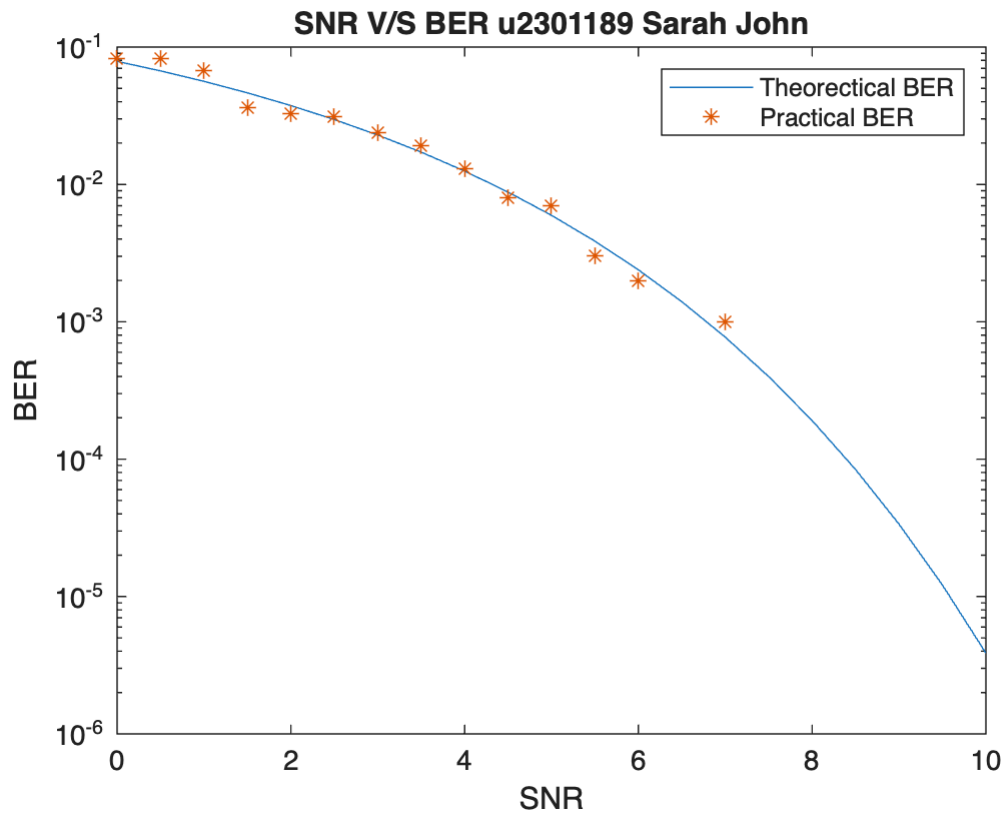
SNR V/S BER plot

```
th_BER=[];
pr_BER=[];
SNR_array=[];
for SNR=0:0.5:10
    SNR_array=[SNR_array SNR];
    No=1/10^(0.1*SNR);
    t_BER=qfunc(sqrt(2/No));
    th_BER=[th_BER t_BER];
    var=No/2;
    std_dev=sqrt(var);
    Noise=std_dev*complex(randn(1, length(qpsk)), randn(1, length(qpsk)));
    qpsk_channel=qpsk+Noise;
    qprb1=reshape(qpsk_channel, [n, N/2]);
    product3=cnorm1*qprb1;
    deconst3=[];
    for k=1:N/2
        if product3(k)>0
            deconst3=[deconst3 1];
        elseif product3(k)<0
            deconst3=[deconst3 0];
        end
    end
    product4=cnorm2*qprb1;
    deconst4=[];
    for k=1:N/2
        if product4(k)>0
            deconst4=[deconst4 1];
        elseif product4(k)<0
            deconst4=[deconst4 0];
        end
    end
    reconst=[];
    for j=1:N/2
        reconst=[reconst deconst4(j) deconst3(j)];
    end
    Error=0;
    for l=1:N
        if bit(l)~=reconst(l)
            Error=Error+1;
        end
    end
    BER=Error/N;
    pr_BER=[pr_BER BER];
end
semilogy(SNR_array, th_BER, SNR_array, pr_BER, "*");
xlabel("SNR")
ylabel("BER")
```

```

legend("Theoretical BER", "Practical BER")
title("SNR V/S BER u2301189 Sarah John")

```



Scatter Plot

```

I_comp = real(cnorm2 * qprb1);
Q_comp = real(cnorm1 * qprb1);
scatter(I_comp, Q_comp, 'filled');
xlabel('In-phase'); ylabel('Quadrature');
title('QPSK Constellation (Scatter Plot) u2301189 Sarah John')

```

QPSK Constellation (Scatter Plot) u2301189 Sarah John

